# Занятие 47.3. Указатели типа void

**Указатель типа void** (или ***«общий указатель»***) — это специальный тип **указателя**, который может указывать на объекты любого типа данных! Объявляется он как обычный указатель, только вместо типа данных используется **ключевое слово void**:

```
void *ptr; // ptr - это указатель типа void
```

Указатель типа void может указывать на объекты любого типа данных:

```
int nResult;
float fResult;

struct Something
{
    int n;
    float f;
};

Something sResult;

void *ptr;
ptr = &nResult; // допустимо
ptr = &fResult; // допустимо
ptr = &sResult; // допустимо
```

Однако, поскольку указатель типа void сам не знает, на объект какого типа он будет указывать, разыменовать его напрямую не получится! Вам сначала нужно будет явно преобразовать указатель типа void с помощью **оператора static_cast** в другой тип данных, а затем уже его разыменовать:

```
#include <iostream>

int main()
{
    int value = 7;
    void *voidPtr = &value;

    //std::cout << *voidPtr << std::endl; // запрещено: нельзя разыменовать указатель типа void

    int *intPtr = static_cast<int*>(voidPtr); // однако, если мы конвертируем наш указатель типа void в указатель типа int,
    std::cout << *intPtr << std::endl; // то мы сможем его разыменовать, будто бы это обычный указатель

    return 0;
}
```

Результат выполнения программы:

```
7
```

Возникает вопрос: «Если указатель типа void сам не знает, на что он указывает, то как мы тогда можем знать, в какой тип данных его следует явно конвертировать с помощью оператора static_cast?». Никак, это уже на ваше усмотрение, вам самим придется выбрать нужный тип. Например:

```
#include <iostream>

enum Type
{
    INT,
    DOUBLE,
    CSTRING
};

void printValue(void *ptr, Type type)
{
    switch (type)
    {
        case INT:
            std::cout << *static_cast<int*>(ptr) << '\n'; // конвертируем в указатель типа int и выполняем разыменование
            break;
        case DOUBLE:
            std::cout << *static_cast<double*>(ptr) << '\n'; // конвертируем в указатель типа double и выполняем разыменование
            break;
        case CSTRING:
            std::cout << static_cast<char*>(ptr) << '\n'; // конвертируем в указатель типа char (без разыменования)
            // std::cout знает, что char* следует обрабатывать как строку C-style.
            // Если бы мы разыменовали результат (целое выражение), то тогда бы вывелся просто первый символ из массива букв,
            break;
    }
}

int main()
{
    int nValue = 7;
    double dValue = 9.3;
    char szValue[] = "Jackie";

    printValue(&nValue, INT);
    printValue(&dValue, DOUBLE);
    printValue(szValue, CSTRING);

    return 0;
}
```

Результат выполнения программы:

```
7
9.3
Jackie
```

Указателям типа void можно присвоить нулевое значение:

```
void *ptr = 0; // ptr - это указатель типа void, который сейчас является нулевым
```

Хотя некоторые компиляторы позволяют удалять указатели типа void, которые указывают на **динамически выделенную память**, делать это не рекомендуется, так как результаты могут быть неожиданными.

Также не получится выполнить **адресную арифметику** с указателями типа void, так как для этого требуется, чтобы указатель знал размер объекта, на который он указывает (для выполнения корректного **инкремента/декремента**). Также нет такого понятия, как **ссылка** на void.

## Заключение

В общем, использовать указатели типа void рекомендуется только в самых крайних случаях, когда без этого не обойтись, так как с их использованием проверку типов данных ни вам, ни компилятору выполнить не удастся. А это, в свою очередь, позволит вам случайно сделать то, что не имеет смысла, и компилятор на это жаловаться не будет. Например:

```
    int nResult = 7;
    printResult(&nResult, CSTRING);
```

Здесь компилятор промолчит. Но что будет в результате? Непонятно!

Хотя код, приведенный выше, кажется аккуратным способом заставить одну функцию обрабатывать несколько типов данных, в языке C++ есть гораздо лучший способ сделать то же самое (через перегрузку функций), в котором сохраняется проверка типов для предотвращения неправильного использования. Также для обработки нескольких типов данных можно использовать шаблоны, которые обеспечивают хорошую проверку типов (но об этом уже на следующих уроках).

Если вам все же придется использовать указатель типа void, то убедитесь, что нет лучшего (более безопасного) способа сделать то же самое, но с использованием других механизмов языка C++!

## Тест

В чём разница между **нулевым указателем** и указателем типа void?

Указатель типа void — это указатель, который может указывать на объект любого типа данных, но он сам не знает, какой это будет тип. Для разыменования указатель типа void должен быть явно преобразован с помощью оператора static_cast в другой тип данных. Нулевой указатель — это указатель, который не указывает на адрес. Указатель типа void может быть нулевым указателем.